

Clase 206 — Análisis de artefactos de Linux

Parte: 9 — Forense digital y respuesta a incidentes · Fuente: NIST SP 800-86 y documentación de systemd/journald 🕒 Duración estimada: 120 min · Nivel: Avanzado

Objetivo

Aprender dónde vive la evidencia en un sistema Linux comprometido: logs de sistema (syslog/journald), historial de shell, cron/systemd timers, cuentas y autenticación, y persistencia común de atacantes. Al terminar podrás reconstruir la actividad de un usuario y de un intruso en Linux.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

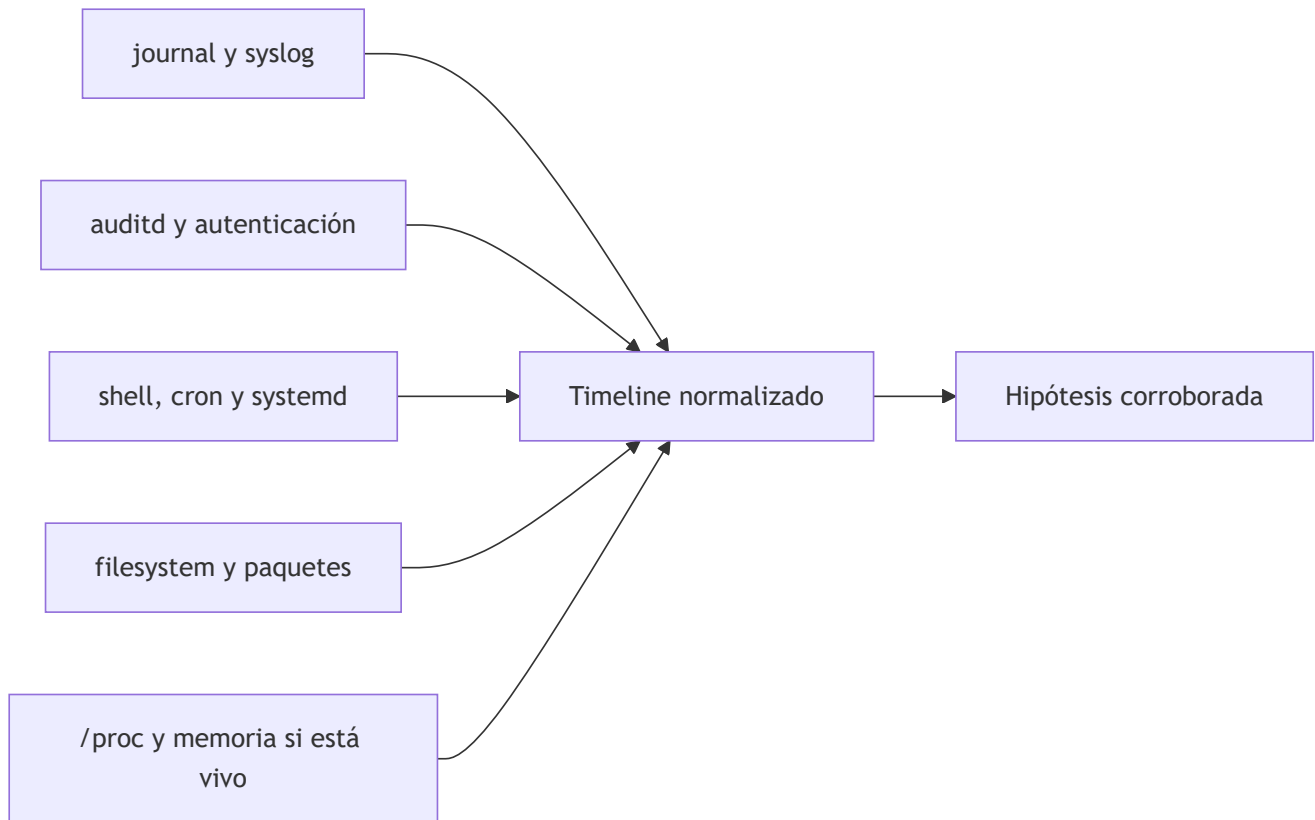
- 1. **Analizar** logs de autenticación y sistema en `/var/log` y `journald`.
- 2. **Reconstruir** actividad de usuario con historiales de shell y timestamps.
- 3. **Detectar** mecanismos de persistencia (cron, systemd, rc, bashrc).
- 4. **Interpretar** cuentas, sudoers y claves SSH sospechosas.
- 5. **Correlacionar** artefactos para trazar una intrusión.

Temas

#	Tema	Por qué importa
1	<code>/var/log</code> y syslog	Registro central del sistema
2	journald (systemd)	Logs binarios modernos
3	auth.log / secure y wtmp/btmp	Autenticación y sesiones
4	Historial de shell	Comandos ejecutados
5	cron y systemd timers	Persistencia programada
6	Cuentas, sudoers, SSH keys	Acceso y escalado
7	Timestamps y <code>stat</code>	Orden de eventos
8	Persistencia típica de atacantes	Qué buscar

Explicación en profundidad

Linux no ofrece un conjunto único de artefactos: distribución, init, filesystem, auditoría, shell y rotación cambian lo disponible. `systemd-journald` puede usar almacenamiento volátil o persistente; syslog puede reenviar y rotar; el historial de shell depende de configuración y momento de cierre.



Se documentan zona horaria, boot ID, hostname y rotaciones antes de ordenar. Un servicio persistente puede aparecer en unit files, symlinks y journal; una cuenta en passwd, shadow, sudoers y logs. `/proc` es volátil y debe adquirirse temprano. La ausencia de history no prueba eliminación deliberada.

Journal, syslog y el límite de la retención

`systemd-journald` añade campos como unidad, PID, UID, boot ID y transporte. Puede almacenar en `/run/log/journal` y perderse al reiniciar, o persistir en `/var/log/journal`, según configuración. `journalctl --list-boots` y filtros por `_BOOT_ID` evitan mezclar reinicios. La verificación del journal ayuda a detectar corrupción cuando existen sellos configurados, pero no demuestra que todo evento relevante se generó.

Syslog y logs de aplicaciones pueden coexistir, duplicarse o reenviarse. Antes de correlacionar se identifica quién originó el mensaje, quién lo escribió y qué regla lo rotó. Un hueco puede corresponder a `logrotate`, límite de tamaño, servicio detenido, reloj incorrecto o manipulación. La copia remota independiente suele aportar mejor resistencia que confiar únicamente en el host investigado.

Sesión, privilegio y comandos

`auth.log` o `secure` depende de la familia de distribución; wtmp y btmp registran sesiones en estructuras binarias, no intención ni todos los comandos. `sudo` puede producir una línea por ejecución según política, mientras `auditd` requiere reglas que definan qué observar. Se relacionan cuenta declarada, UID efectivo, TTY, origen SSH, proceso y resultado.

El historial de shell se escribe según shell y opciones. Bash puede mantener comandos en memoria hasta el cierre; timestamps dependen de `HISTTIMEFORMAT` y el archivo puede truncarse. Por ello un comando

presente es evidencia contextual y uno ausente no descarta ejecución. Process accounting, audit, journal, EDR o filesystem ofrecen corroboración independiente.

Persistencia como relación de activación

Cron, timers, units, scripts de inicio, claves `authorized_keys`, perfiles y cargadores dinámicos pueden iniciar código. La detección no consiste en buscar solo nombres extraños: se identifica **qué activa, con qué identidad, qué ejecuta, desde qué ruta y cuándo cambió**. Una clave SSH nueva puede ser administración autorizada; se compara con gestión de configuración, propietario y origen de acceso antes de llamarla puerta trasera.

Glosario

- **journald**: servicio de eventos de systemd.
- **auditd**: subsistema de auditoría de Linux.
- **Unit file**: definición de servicio o temporizador systemd.
- **Cron**: programación tradicional de tareas.
- **/proc**: vista efímera del kernel y procesos.
- **Log rotation**: archivado y reemplazo periódico de logs.
- **Boot ID**: identificador de arranque para contextualizar eventos.

Definiciones y características

- **syslog**: sistema clásico de logs de texto en `/var/log`. Característica: legible con herramientas estándar.
- **journald**: servicio de journal de systemd, consultado con `journalctl`. Característica: añade metadatos estructurados, pero su persistencia, sellado y retención dependen de configuración.
- **wtmp/btmp/lastlog**: registros binarios de logins exitosos, fallidos y último acceso. Característica: se leen con `last`, `lastb`, `lastlog`.
- **Historial de shell**: archivos como `~/.bash_history` o `~/.zsh_history`. Característica: escritura y timestamps dependen del shell, opciones y cierre de sesión.
- **cron / systemd timer**: tareas programadas. Característica: vías comunes de persistencia.
- **authorized_keys**: claves públicas autorizadas para autenticación SSH, sujetas a opciones y política del servidor. Característica: una adición no autorizada puede crear persistencia, pero una clave nueva también puede ser administración legítima.
- **stat**: muestra timestamps atime/mtime/ctime de un archivo. Característica: base del timeline en Linux.

Caso razonado — persistencia después de un acceso SSH

Una cuenta de despliegue inicia sesión desde una IP no habitual. `journalctl` y `auth.log` coinciden en usuario y origen; `wtmp` confirma una sesión, pero no sus comandos. El inode y `stat` muestran cambio en `authorized_keys`; una nueva unit de usuario activa un script desde `/tmp`. El journal contiene el arranque de la unit después del login.

La hipótesis se fortalece por la secuencia, no por una clave aislada. Se conserva el boot ID, se verifica zona y se compara la clave con el inventario. Si el historial está vacío, el informe no afirma limpieza: indica que la

fuentes no aportan comandos y usa audit, journal y filesystem para reconstruir. La cuenta, la unit y el script se adquieren antes de contención según el plan.

✓ Criterio de dominio

El alumno correlaciona al menos una fuente de autenticación, una de ejecución/persistencia y una de filesystem; explica rotación, boot ID y límites del historial; y distingue cambio observado de intención atribuida. Una búsqueda de `/var/log` sin considerar distribución y configuración no acredita dominio.

🧰 Herramientas y preparación

- **Nativas:** `journalctl`, `last`, `lastb`, `grep`, `stat`, `ausearch` (auditd).
- **Análisis:** The Sleuth Kit para montar la imagen, `log2timeline` /plaso para timeline.
- **Entorno:** usa una imagen de una VM Linux propia. Monta en solo lectura:

```
bash mount -o ro,noexec,nodev imagen.dd /mnt/evidencia
```

🔬 Laboratorio guiado

Trabaja sobre una imagen de una VM Linux propia montada en solo lectura.

1. Revisa autenticación (SSH, sudo):

```
bash grep -Ei "accepted|failed|sudo" /mnt/evidencia/var/log/auth.log
```

2. Lee logins con los registros binarios:

```
bash last -f /mnt/evidencia/var/log/wtmp lastb -f /mnt/evidencia/var/log/btmp
```

3. Consulta journald offline:

```
bash journalctl --directory=/mnt/evidencia/var/log/journal --no-pager
```

4. Extrae el historial de shell de cada usuario:

```
bash cat /mnt/evidencia/home/*/.bash_history cat /mnt/evidencia/root/.bash_history
```

5. Busca persistencia programada:

```
bash ls -la /mnt/evidencia/etc/cron* cat /mnt/evidencia/var/spool/cron/crontabs/* ls -la /mnt/evidencia/etc/systemd/system/
```

6. Revisa cuentas y accesos sospechosos:

```
bash cat /mnt/evidencia/etc/passwd | awk -F: '$3 ≥ 1000' cat /mnt/evidencia/etc/sudoers cat /mnt/evidencia/home/*/.ssh/authorized_keys
```

7. Revisa persistencia en perfiles de shell:

```
bash grep -R . /mnt/evidencia/home/*/.bashrc /mnt/evidencia/etc/rc.local 2>/dev/null
```

8. Ordena hallazgos por tiempo con `stat` sobre archivos sospechosos.

Ejercicios

1. Extrae de `auth.log` todos los logins SSH exitosos y su IP de origen.
2. Detecta un ataque de fuerza bruta contando fallos en `btmpt`.
3. Encuentra una tarea cron maliciosa que tú mismo plantaste.
4. Identifica una cuenta con UID 0 distinta de `root`.
5. Explica la diferencia entre `atime`, `mtime` y `ctime`.
6. Reconstruye la secuencia de comandos de un usuario desde su historial.

Reto verificable

En una VM Linux propia, simula una intrusión (crea un usuario extra con UID 0, añade una clave SSH y un cron de persistencia) y luego, desde la imagen, detecta y documenta los tres mecanismos.

Criterio de aceptación: tu informe identifica el usuario malicioso, la clave SSH añadida y la tarea de persistencia, cada uno con la ruta del artefacto, el timestamp relevante y el comando que lo reveló.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>.bash_history</code> vacío	Atacante lo borró o usó <code>unset HISTFILE</code> . Busca en <code>journald</code> o <code>timeline</code> .
<code>journalctl</code> no lee la carpeta	Falta <code>--directory</code> correcto o <code>journal</code> corrupto. Verifica la ruta.
Timestamps alterados	El atacante usó <code>touch</code> . Contrasta con <code>ctime</code> y con logs.
Montaste con <code>exec</code>	Riesgo de ejecutar malware. Usa siempre <code>ro,noexec,nodev</code> .
No ves los logins	Logs rotados/comprimidos. Descomprime <code>auth.log.*.gz</code> .

Preguntas frecuentes

 **¿journald reemplaza a syslog?** Coexisten. `journald` es el estándar en `systemd`; muchos sistemas también reenvían a `syslog` en texto.

 **¿El historial de shell tiene fechas?** Solo si `HISTTIMEFORMAT` estaba configurado. Si no, tienes el orden pero no la hora exacta.

 **¿Cómo detecto persistencia?** Revisa `cron`, `systemd timers/services`, `rc.local`, perfiles de shell, y `authorized_keys`. Compara contra un baseline limpio.

 **¿ctime se puede falsificar?** `mtime` y `atime` sí con `touch`; `ctime` es más difícil (requiere manipular el reloj o el FS), por eso es más confiable.

Referencias verificables y alcance

- NIST SP 800-86: fuente primaria del proceso de colección y análisis de datos de sistema operativo; no especifica artefactos de cada distribución actual — <https://doi.org/10.6028/NIST.SP.800-86>

- systemd `journalctl` : documentación oficial de filtros, boots, campos y verificación; la disponibilidad depende de versión y configuración — <https://www.freedesktop.org/software/systemd/man/latest/journalctl.html>
- systemd `journald.conf` : documentación oficial de almacenamiento, límites, forwarding y sellado — <https://www.freedesktop.org/software/systemd/man/latest/journald.conf.html>
- OpenSSH `sshd` : manual primario de `AuthorizedKeysFile` y opciones de autenticación — https://man.openbsd.org/sshd_config
- Linux Kernel, ext4: fuente primaria para interpretar inodos y tiempos del filesystem usado en la clase — <https://www.kernel.org/doc/html/latest/filesystems/ext4/index.html>
- The Sleuth Kit: documentación primaria de análisis de filesystem — <https://www.sleuthkit.org/sleuthkit/docs.php>



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

[Clase 205 — Análisis de artefactos de Windows](#)



Siguiente clase

[Clase 207 — Forense de memoria RAM con Volatility](#)